

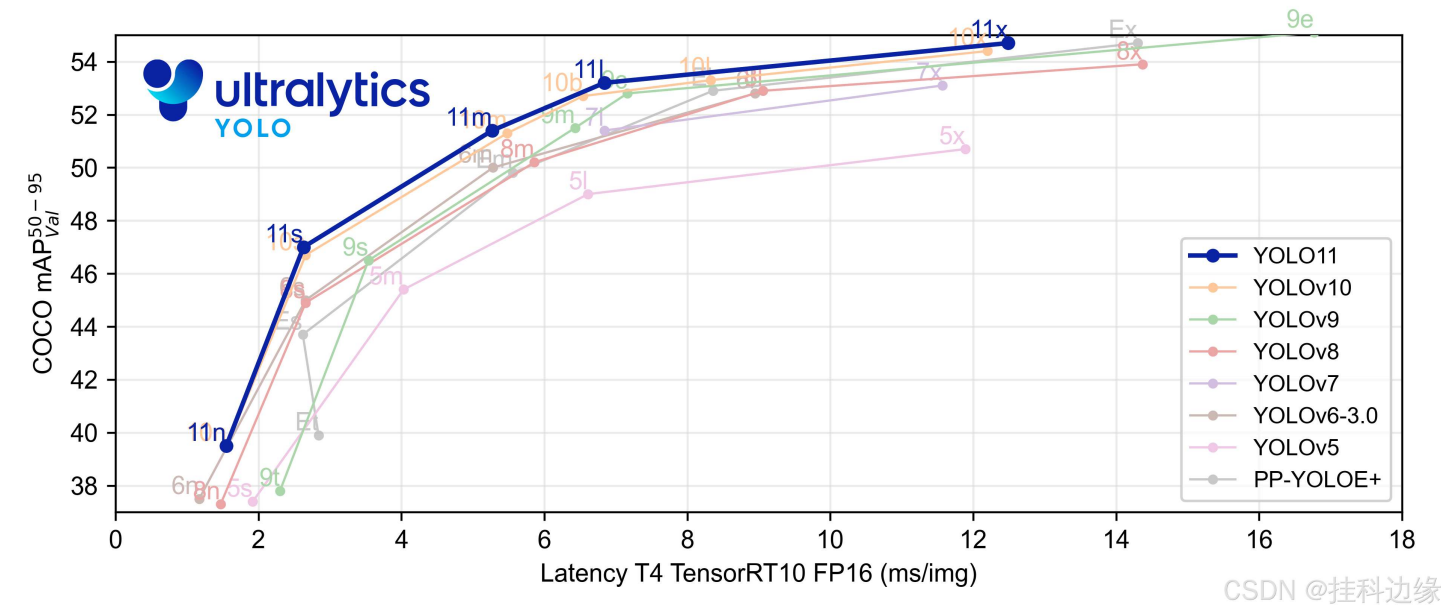
YOLOv11来了，使用YOLOv11训练自己的数据集和推理(附YOLOv11网络结构图)

文章目录

- 前言
- 一、YOLOv11代码下载地址
 - 1.YOLOv11模型结构图
- 二、数据集准备
 - 1.数据集标注软件
 - 2.voc数据集格式转换
 - 3.数据集划分
 - 4.修改yolo的训练配置文件
- 三、YOLO环境配置教程
 - 1.pytorch环境安装
 - 2.其他依赖安装
- 四、YOLOv11训练
- 五、YOLOv11推理
- 六、解决训练过程中断怎么继续上次训练
- 总结

前言

YOLOv11 由 Ultralytics 团队在 2024 年 9 月 30 日发布，最新的 YOLOv11 模型 在之前的 YOLO 版本引入了新功能和改进，以进一步提高性能和 灵活性 。YOLOv11 在快速、准确且易于使用，使其成为各种目标检测和跟踪、实例分割、图像分类和姿态估计任务的绝佳选择。可以看出官网 YOLOv11 在COCO数据集上的性能表现，如下图所示：



CSDN @挂科边缘

一、YOLOv11代码下载地址

手把手视频安装可以看这个视频： [视频安装教程](#)



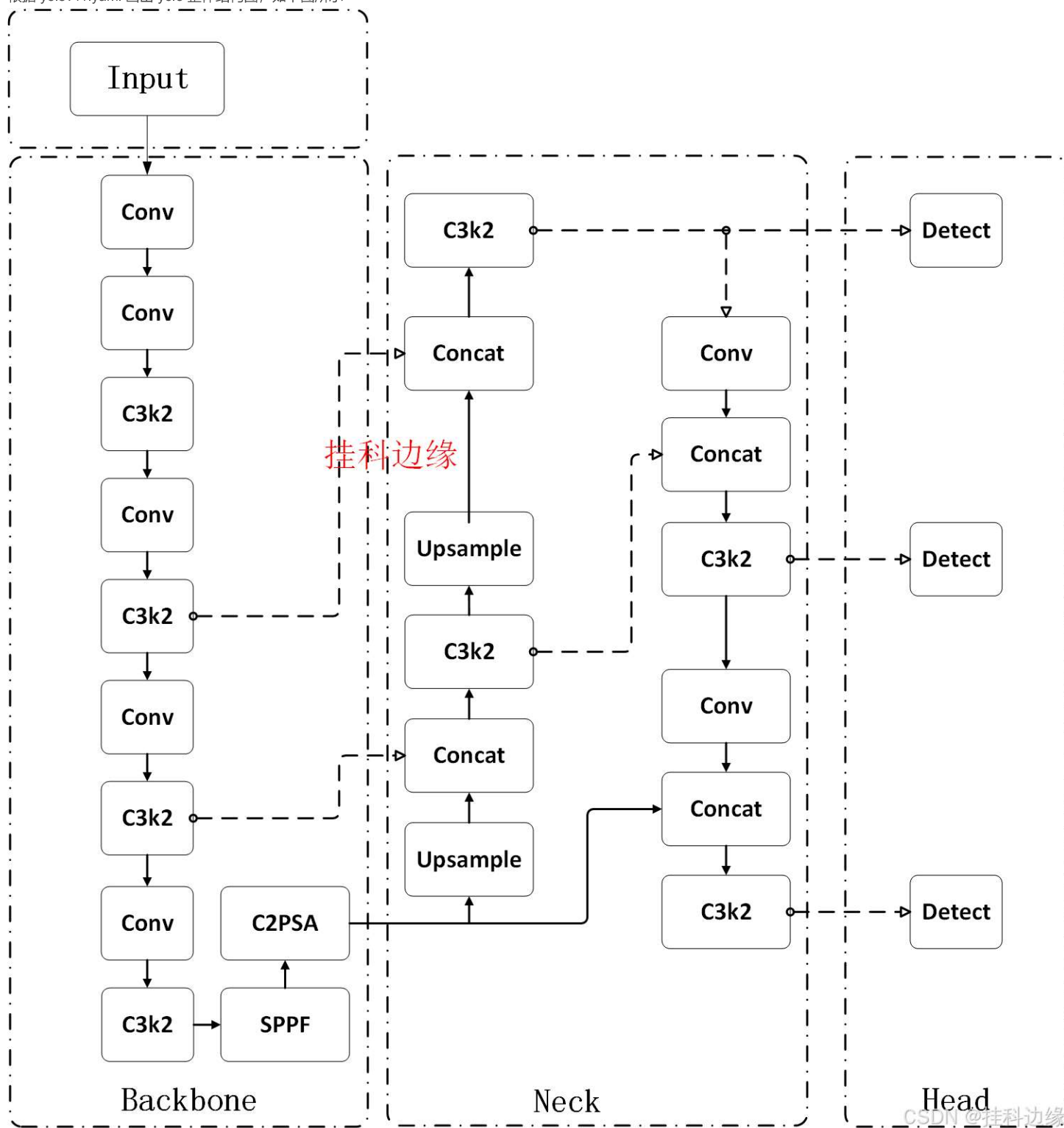
官网的源码下载地址： [官网源码](#)

官网打不开的话，从我的网盘下载就行，链接: [我下载好的源码](#)

提取码: eb9i

1.YOLOv11模型结构图

根据 yolov11.yaml 画出 yolo 整体结构图，如下图所示

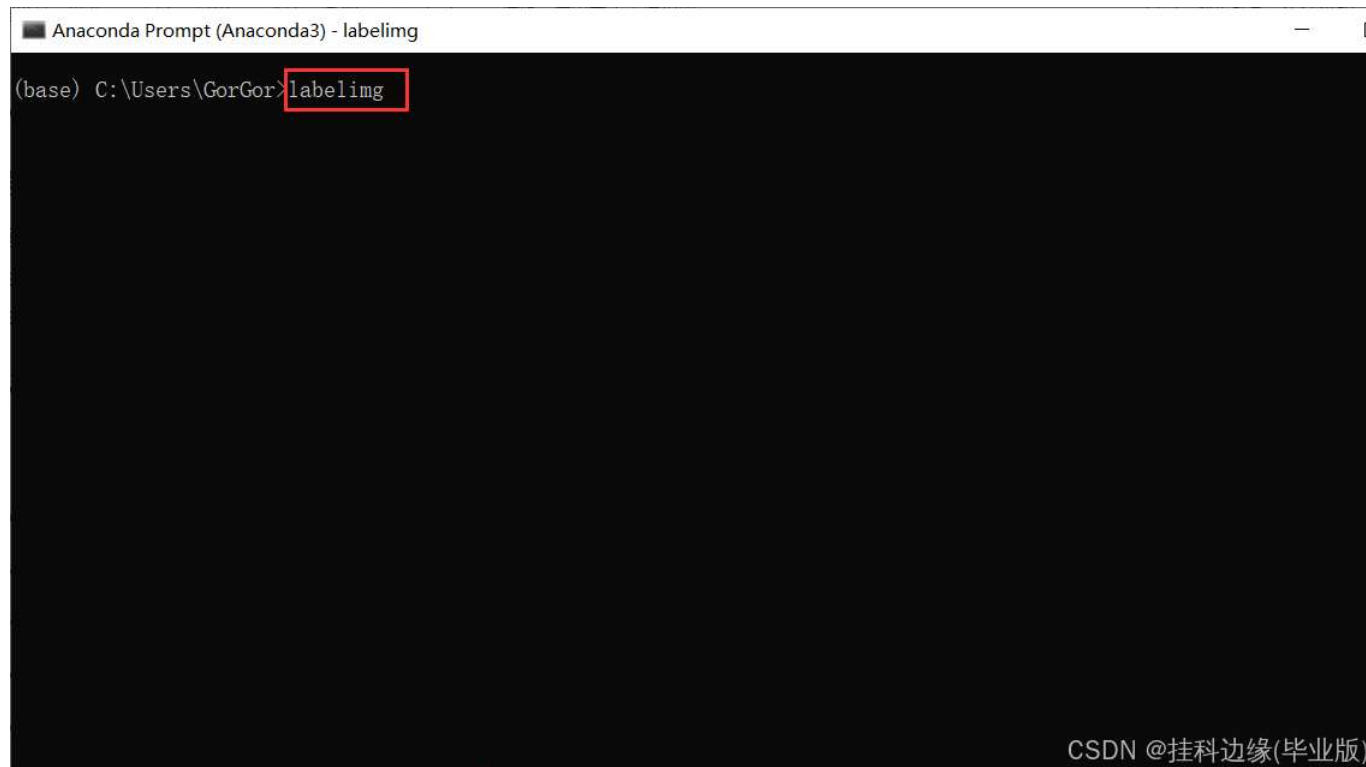


二、数据集准备

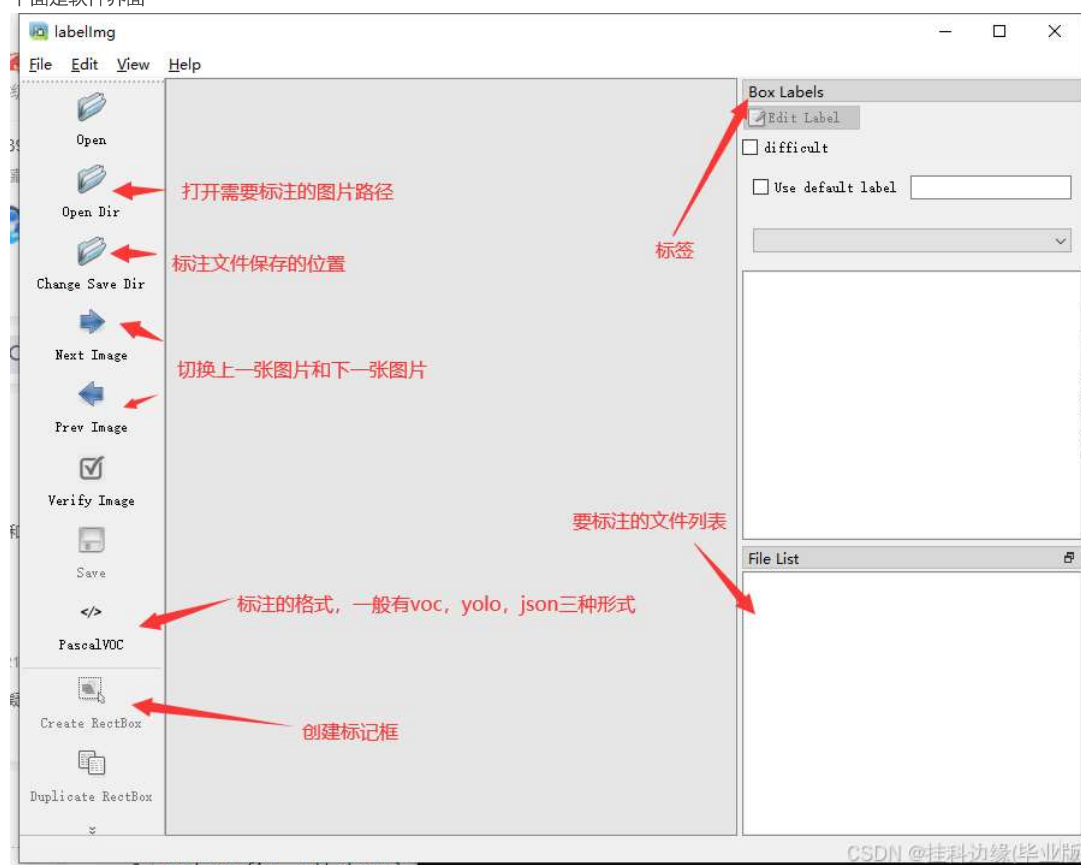
1. 数据集标注软件

数据集使用标注软件标注好，我这里推荐两个标注软件，一个是 [labelimg](#)，另外一个 [labelme](#)，可以在python环境，使用 `pip install labelimg` 或者 `pip install labelme` 进行安装，看你选择哪个标注工具标注了，我使用 [labelimg](#) 标注工具

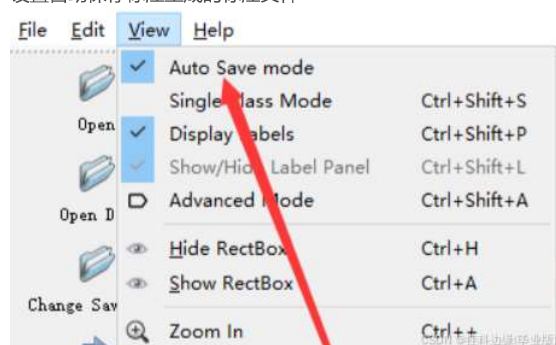
安装完成后在终端输入命令启动标注软件



下面是软件界面



设置自动保存标注生成的标注文件



2.voc数据集格式转换

标注格式如果**选择VOC格式**，后面需要代码转换格式，如果选择yolo格式就不用转换，voc格式转换yolo格式代码如下：

```
1 import xml.etree.ElementTree as ET
2 import os, cv2
3 import numpy as np
4 from os import listdir
5 from os.path import join
6
7 classes = []
8
9 def convert(size, box):
10     dw = 1. / (size[0])
11     dh = 1. / (size[1])
12     x = (box[0] + box[1]) / 2.0 - 1
13     y = (box[2] + box[3]) / 2.0 - 1
14     w = box[1] - box[0]
15     h = box[3] - box[2]
16     x = x * dw
17     w = w * dw
18     y = y * dh
19     h = h * dh
20     return (x, y, w, h)
21
22
23 def convert_annotation(xmlpath, xmlname):
24     with open(xmlpath, "r", encoding='utf-8') as in_file:
25         txtname = xmlname[:-4] + '.txt'
26         txtfile = os.path.join(txtpath, txtname)
27         tree = ET.parse(in_file)
28         root = tree.getroot()
29         filename = root.find('filename')
30         img = cv2.imdecode(np.fromfile('{}/{}'.format(imgpath, xmlname[:-4], postfix), np.uint8), cv2.IMREAD_COLOR)
31         h, w = img.shape[:2]
32         res = []
33         for obj in root.iter('object'):
34             cls = obj.find('name').text
35             if cls not in classes:
36                 classes.append(cls)
37                 cls_id = classes.index(cls)
38                 xmlbox = obj.find('bndbox')
39                 b = (float(xmlbox.find('xmin').text), float(xmlbox.find('xmax').text), float(xmlbox.find('ymin').text),
40                     float(xmlbox.find('ymax').text))
41                 bb = convert((w, h), b)
42                 res.append(str(cls_id) + " " + " ".join([str(a) for a in bb]))
43         if len(res) != 0:
44             with open(txtfile, 'w+') as f:
45                 f.write('\n'.join(res))
46
47
48 if __name__ == "__main__":
49     postfix = 'png' # 图像后缀
50     imgpath = r'E:\A-毕业设计代做数据\helmet\test\images' # 图像文件路径
51     xmlpath = r'E:\A-毕业设计代做数据\helmet\test\annotations' # xml文件文件路径
52     txtpath = r'E:\A-毕业设计代做数据\helmet\test\labels' # 生成的txt文件路径
53
54     if not os.path.exists(txtpath):
55         os.makedirs(txtpath, exist_ok=True)
56
57     list = os.listdir(xmlpath)
58     error_file_list = []
59     for i in range(0, len(list)):
60         try:
61             path = os.path.join(xmlpath, list[i])
62             if ('.xml' in path) or ('.XML' in path):
63                 convert_annotation(path, list[i])
64                 print(f'file {list[i]} convert success.')
65             else:
66                 print(f'file {list[i]} is not xml format.')
67         except Exception as e:
68             print(f'file {list[i]} convert error.')
69             print(f'error message:\n{e}')
70             error_file_list.append(list[i])
71     print(f'this file convert failure\n{error_file_list}')
72     print(f'Dataset Classes:{classes}')
```

代码需要修改的地方如下:

- 1.postfix参数填图片的后缀, 需要注意图片格式要统一, 是png格式就写png, 是jpg格式就写jpg
- 2.imgpath参数填图片所在的路径
- 3.xmlpath参数填标注文件的路径
- 4.txtpath参数填生成的yolo格式的文件

```
if __name__ == "__main__":
    postfix = 'png' # 图像后缀
    imgpath = r'E:\A-毕业设计代做数据\helmet\test\images' # 图像文件路径
    xmlpath = r'E:\A-毕业设计代做数据\helmet\test\annotations' # xml文件文件路径
    txtpath = r'E:\A-毕业设计代做数据\helmet\test\labels' # 生成的txt文件路径
CSDN @挂科边缘(毕业版)
```

3.数据集划分

划分训练集和验证集代码如下:

```
1 import os, shutil
2 from sklearn.model_selection import train_test_split
3
4
5 val_size = 0.2
6 #test_size = 0.2
7 postfix = 'jpg'
8 imgpath = r'E:\A-毕业设计代做数据\datasets\images'
9 txtpath = r'E:\A-毕业设计代做数据\datasets\labels'
10
11
12
13 output_train_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\train'
14 output_val_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\val'
15 output_train_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\train'
16 output_val_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\val'
17
18 os.makedirs(output_train_img_folder, exist_ok=True)
19 os.makedirs(output_val_img_folder, exist_ok=True)
20 os.makedirs(output_train_txt_folder, exist_ok=True)
21 os.makedirs(output_val_txt_folder, exist_ok=True)
22
23
24 listdir = [i for i in os.listdir(txtpath) if 'txt' in i]
25 train, val = train_test_split(listdir, test_size=val_size, shuffle=True, random_state=0)
26
27 #todo: 需要test放开
28
29 # train, test = train_test_split(listdir, test_size=test_size, shuffle=True, random_state=0)
30 # train, val = train_test_split(train, test_size=val_size, shuffle=True, random_state=0)
31
32 for i in train:
33     img_source_path = os.path.join(imgpath, '{}.{}'.format(i[:-4], postfix))
34     txt_source_path = os.path.join(txtpath, i)
35
36     img_destination_path = os.path.join(output_train_img_folder, '{}.{}'.format(i[:-4], postfix))
37     txt_destination_path = os.path.join(output_train_txt_folder, i)
38
39     shutil.copy(img_source_path, img_destination_path)
40     shutil.copy(txt_source_path, txt_destination_path)
41
42 for i in val:
43     img_source_path = os.path.join(imgpath, '{}.{}'.format(i[:-4], postfix))
44     txt_source_path = os.path.join(txtpath, i)
45
46     img_destination_path = os.path.join(output_val_img_folder, '{}.{}'.format(i[:-4], postfix))
47     txt_destination_path = os.path.join(output_val_txt_folder, i)
48
49     shutil.copy(img_source_path, img_destination_path)
50     shutil.copy(txt_source_path, txt_destination_path)
51
52
53 #
54 # for i in train:
55
```

```

56 # shutil.copy('{}\{}\{}'.format(imgpath, i[:-4], postfix), r'E:\1-cheng\4-yolo-dataset-daizuo\multi-classify\bird-boat-horse-aerop
57 # shutil.copy('{}\{}\{}'.format(txtpath, i), r'E:\1-cheng\4-yolo-dataset-daizuo\multi-classify\bird-boat-horse-aeroplane-sheep\datase
58 #
59 # for i in val:
60 # shutil.copy('{}\{}\{}'.format(imgpath, i[:-4], postfix), r'E:\1-cheng\4-yolo-dataset-daizuo\multi-classify\bird-boat-horse-aerop
61 # shutil.copy('{}\{}\{}'.format(txtpath, i), r'E:\1-cheng\4-yolo-dataset-daizuo\multi-classify\bird-boat-horse-aeroplane-sheep\datase
62
63 #todo:需要test则放开
64
65 # for i in test:
66 # shutil.copy('{}\{}\{}'.format(imgpath, i[:-4], postfix), 'images/test/\{}\{}'.format(i[:-4], postfix))
67 # shutil.copy('{}\{}\{}'.format(txtpath, i), 'labels/test/\{}\{}'.format(i))

```

需要修改的地方如下

```

7 postfix = 'jpg'
8 imgpath = r'E:\A-毕业设计代做数据\datasets\images'
9 txtpath = r'E:\A-毕业设计代做数据\datasets\labels'
10
11
12
13 output_train_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\train'
14 output_val_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\val'
15 output_train_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\train'
16 output_val_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\val'
17

```

postfix填图片后缀
imgpath填图片路径
txtpath填txt文件

CSDN @挂科边缘(毕业版)

下面四个参数只需在自己电脑任意位置新建一个文件夹就行，用于存放生成的训练集和验证集，比如新建一个文件夹叫dataset_kengwa，后面的路径不用动，如下图左边的框出来的路径覆盖成你的就行

```

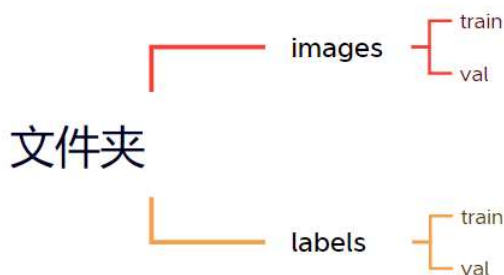
output_train_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\train'
output_val_img_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\images\val'
output_train_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\train'
output_val_txt_folder = r'E:\A-毕业设计代做数据\datasets\dataset_kengwa\labels\val'

```

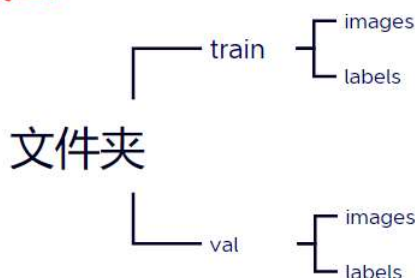
CSDN @挂科边缘(毕业版)

数据集有以下两种方式放置，都可以进行训练，常见的数据集放置是第一种，也有 [开源](#) 的数据集按照第二种方式放置的，我都遇见过，也能训练起来

方式一：

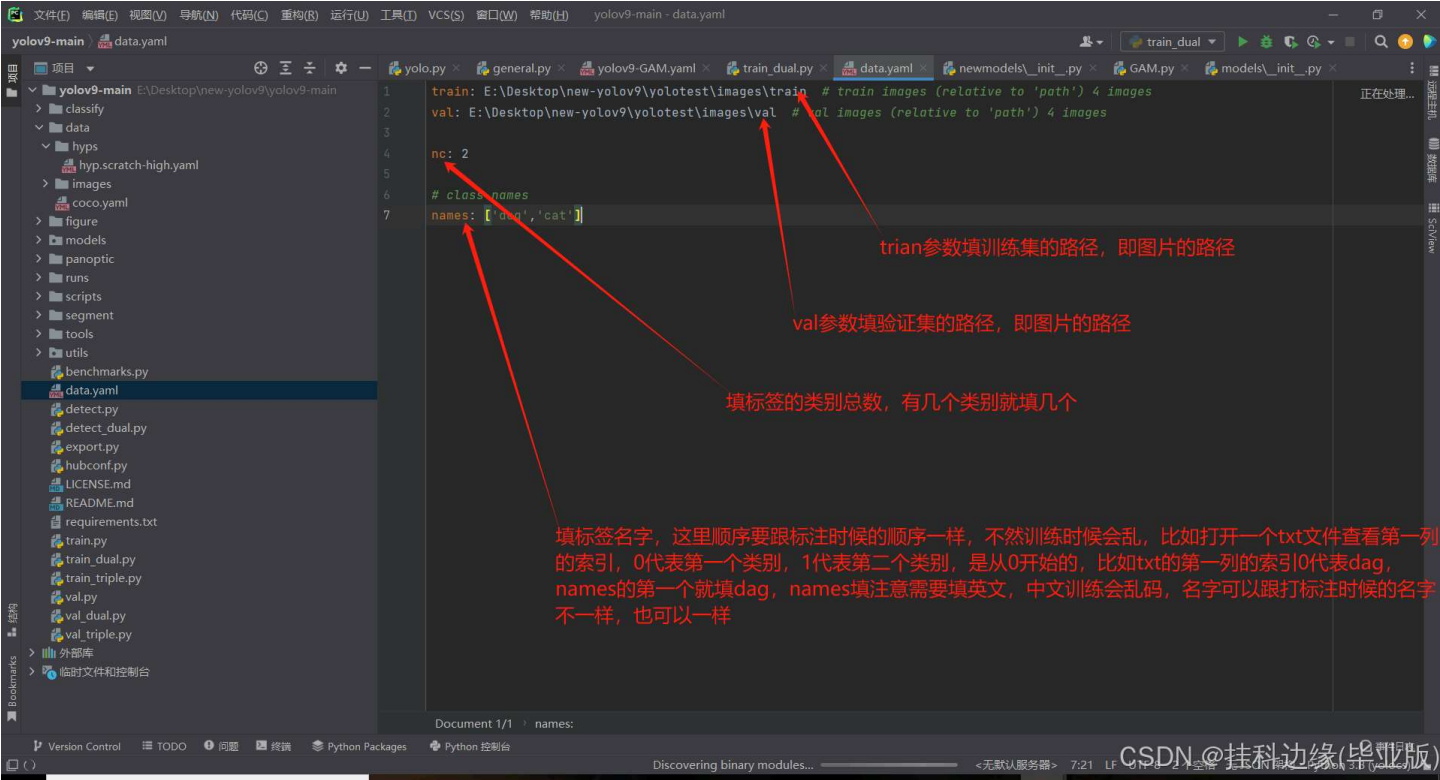


方式二：



4.修改yolo的训练配置文件

我们需要在项目下创建一个data.yaml的文件，文件名根据数据集名称取，我这里方便演示直接叫data.yaml，如下图所示



代码如下：

```
1 | train: E:\Desktop\new-yolov9\yolotest\images\train # train images (relative to 'path') 4 images
2 | val: E:\Desktop\new-yolov9\yolotest\images\val # val images (relative to 'path') 4 images
3 |
4 | nc: 2
5 |
6 | # class names
7 | names: ['dog', 'cat']
```

三、YOLO环境配置教程

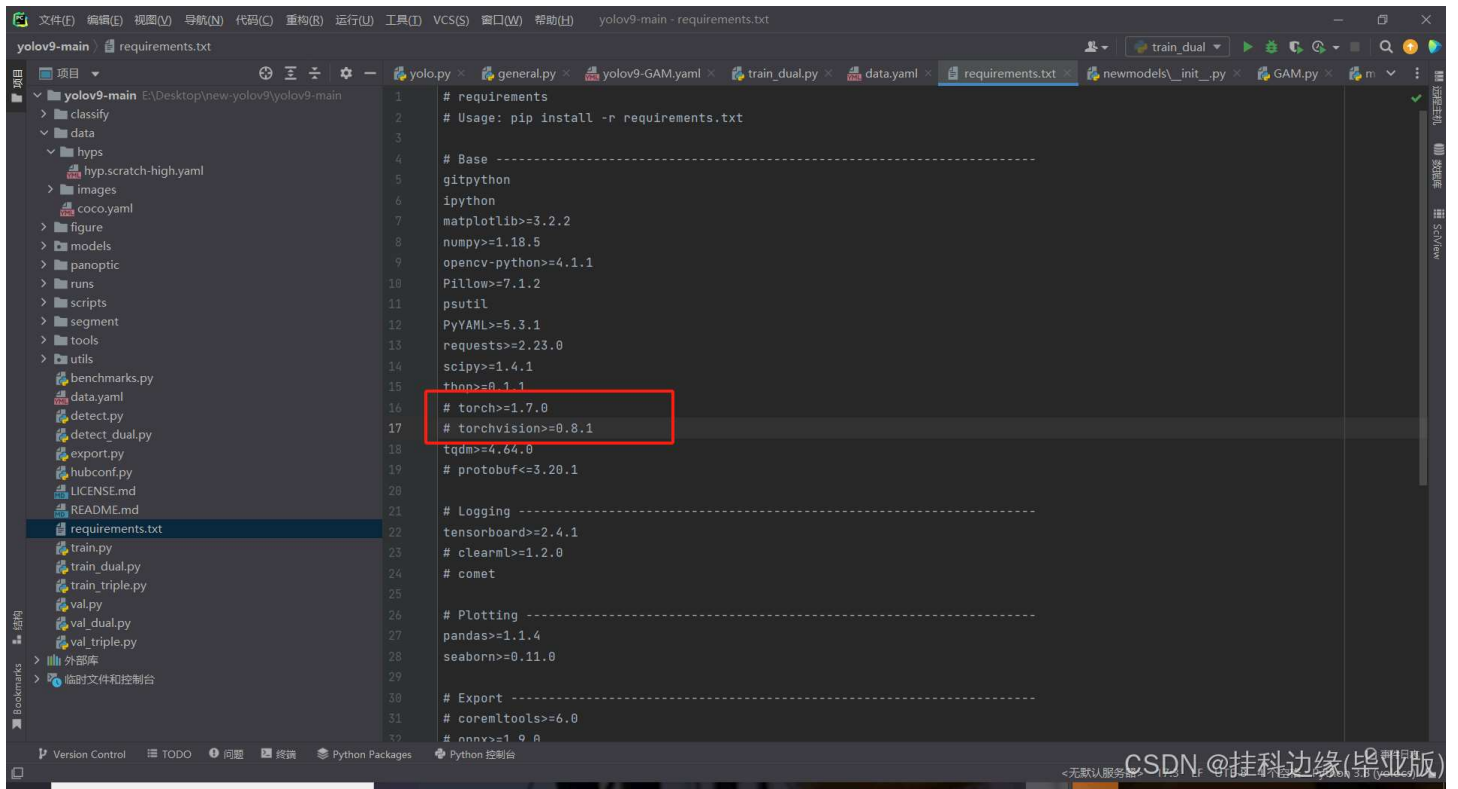
YOLOv11/YOLOv10/YOLOv9/YOLOv8 /YOLOv7 /YOLOv5 环境都是通用的，只需要安装一次就行

1.pytorch环境安装

手把书pytorch安装教程[链接参考链接](#): pytorch环境安装教程视频

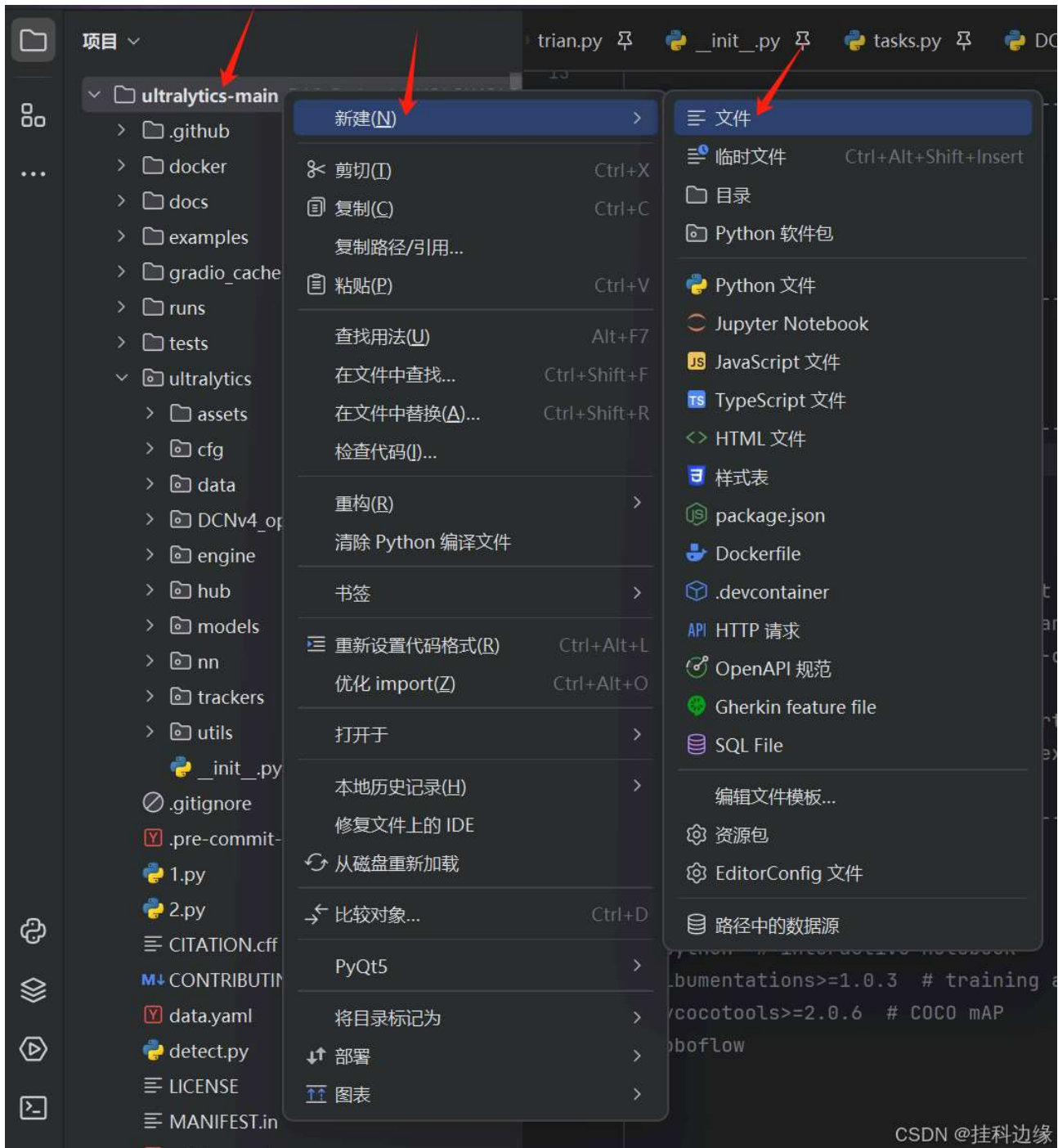
2.其他依赖安装

安装requirements.txt文件的环境，需要注释掉下面两行，前面的步骤已经安装了，不注释的话会覆盖前面的会安装最新版本的pytorch，所以注释掉



```
1 # requirements
2 # Usage: pip install -r requirements.txt
3
4 # Base -----
5 gitpython
6 ipython
7 matplotlib>=3.2.2
8 numpy>=1.18.5
9 opencv-python>=4.1.1
10 Pillow>=7.1.2
11 psutil
12 PyYAML>=5.3.1
13 requests>=2.23.0
14 scipy>=1.4.1
15 thon>=8.1.1
16 # torch>=1.7.0
17 # torchvision>=0.8.1
18 tqdm>=4.64.0
19 # protobuf<=3.20.1
20
21 # Logging -----
22 tensorboard>=2.4.1
23 # clearml>=1.2.0
24 # comet
25
26 # Plotting -----
27 pandas>=1.1.4
28 seaborn>=0.11.0
29
30 # Export -----
31 # coremltools>=6.0
32 # onnx>=1.9.0
```

没有这个文件可以自己新建一个requirements.txt，然后把下面代码复制进去就好了



CSDN @挂科边缘

新建文件

requirements.txt

CSDN @挂科边缘

```
1 # Ultralytics requirements
2 # Example: pip install -r requirements.txt
3
4 # Base -----
5 matplotlib>=3.3.0
6 numpy==1.24.4 # pinned by Snyk to avoid a vulnerability
7 opencv-python>=4.6.0
8 pillow>=7.1.2
9 pyyaml>=5.3.1
10 requests>=2.23.0
11 scipy>=1.4.1
12 tqdm>=4.64.0
13
14 # Logging -----
15 # tensorboard>=2.13.0
16 # dvclive>=2.12.0
17 # clearml
18 # comet
19
```

```

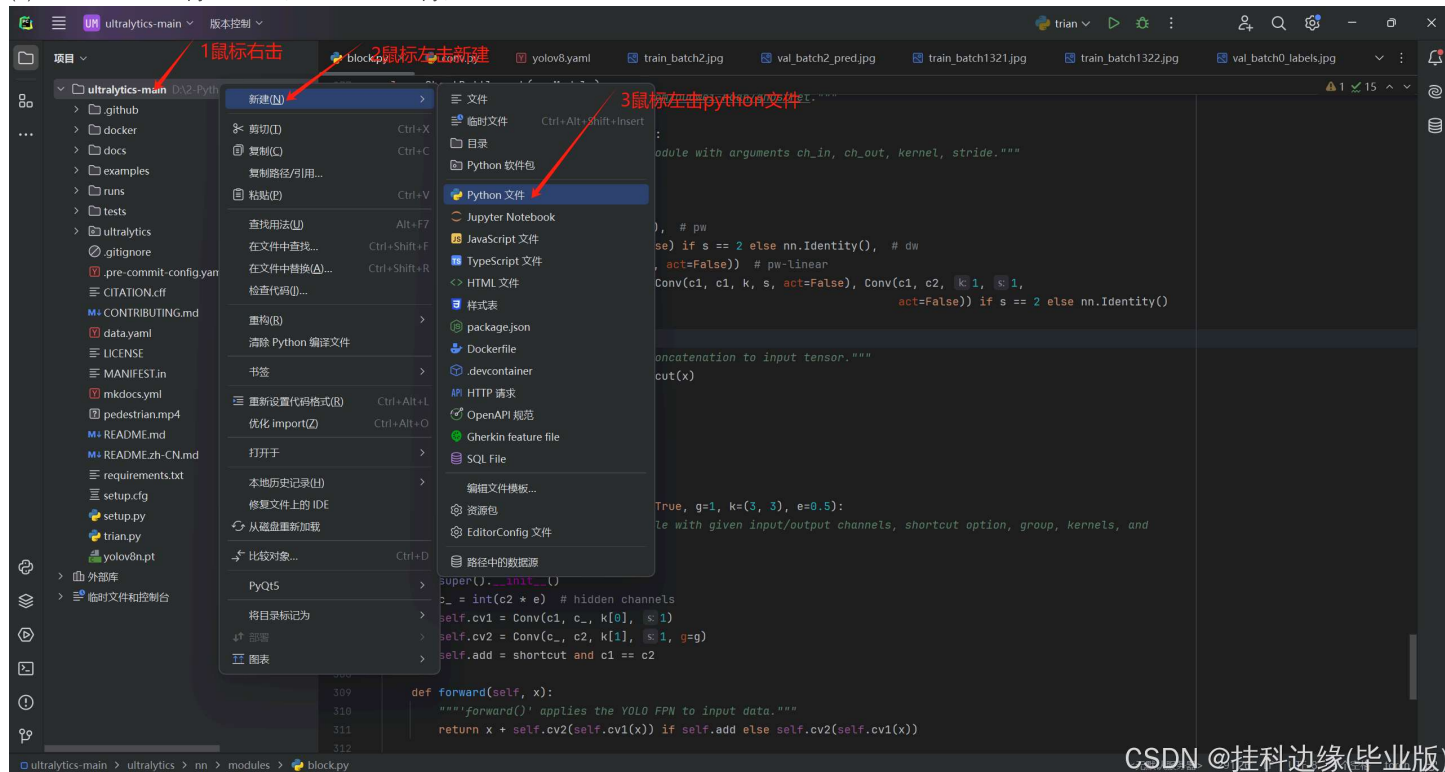
20 # Plotting -----
21 pandas>=1.1.4
22 seaborn>=0.11.0
23
24 # Export -----
25 # coremltools>=7.0 # CoreML export
26 # onnx>=1.12.0 # ONNX export
27 # onnxsim>=0.4.1 # ONNX simplifier
28 # nvidia-pyindex # TensorRT export
29 # nvidia-tensorrt # TensorRT export
30 # scikit-learn==0.19.2 # CoreML quantization
31 # tensorflow>=2.4.1 # TF exports (-cpu, -aarch64, -macos)
32 # tflite-support
33 # tensorflowjs>=3.9.0 # TF.js export
34 # openvino-dev>=2023.0 # OpenVINO export
35
36 # Extras -----
37 psutil # system utilization
38 py-cpuinfo # display CPU info
39 thop>=0.1.1 # FLOPs computation
40 # ipython # interactive notebook
41 # albumentations>=1.0.3 # training augmentations
42 # pycocotools>=2.0.6 # COCO mAP
43 # roboflow
44

```

手把书安装教程链接参考链接：其他依赖库安装，视频教程

四、YOLOv11训练

(1)在根目录新建一个python文件，取名为：train.py，如果之前看过我的文章，已经新建过就不用重新新建了



(2)把训练代码复制到train.py文件，如果之前看过我的文章，已经复制过了就不用重新复制了，只需修改参数就行训练的代码如下：

```

1 # -*- coding: utf-8 -*-
2 """
3 @Auth : 挂科边缘
4 @File : train.py
5 @IDE : PyCharm
6 @Motto:学习新思想，争做新青年
7 @Email : 179958974@qq.com
8

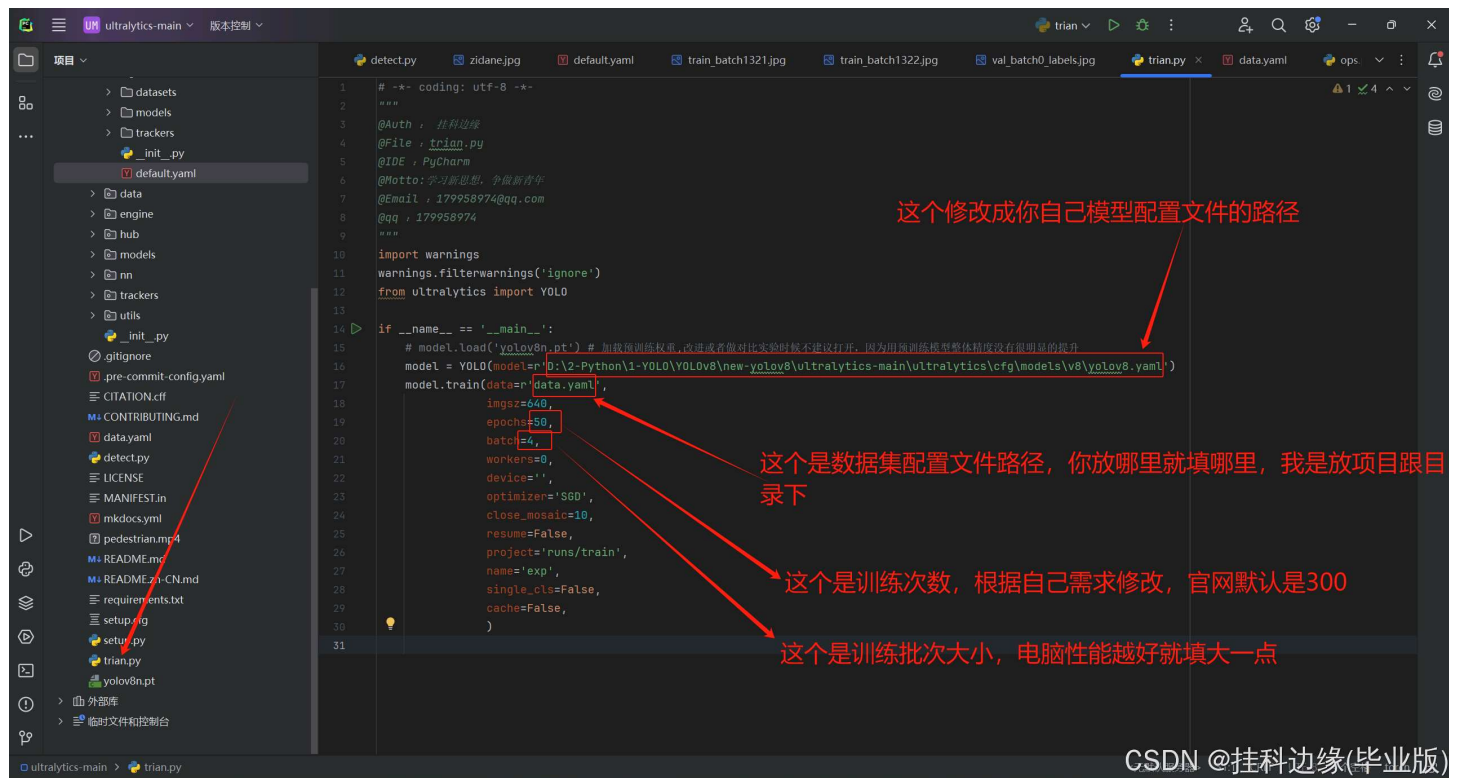
```

```

9 """
10 import warnings
11 warnings.filterwarnings('ignore')
12 from ultralytics import YOLO
13
14 if __name__ == '__main__':
15     # model.load('yolov8n.pt') # 加载预训练权重,改进或者做对比实验时候不建议打开,因为用预训练模型整体精度没有很明显的提升
16     model = YOLO(model='D:\2-Python\1-YOLO\YOLOv11\ultralytics-8.3.2\ultralytics\cfg\models\11\yolov11.yaml')
17     model.train(data='data.yaml',
18                 imgs=640,
19                 epochs=50,
20                 batch=4,
21                 workers=0,
22                 device='',
23                 optimizer='SGD',
24                 close_mosaic=10,
25                 resume=False,
26                 project='runs/train',
27                 name='exp',
28                 single_cls=False,
29                 cache=False,
30                 )

```

注意注意注意: 模型配置路径改成你自己的路径, 还有数据集配置文件也修改成你自己的路径



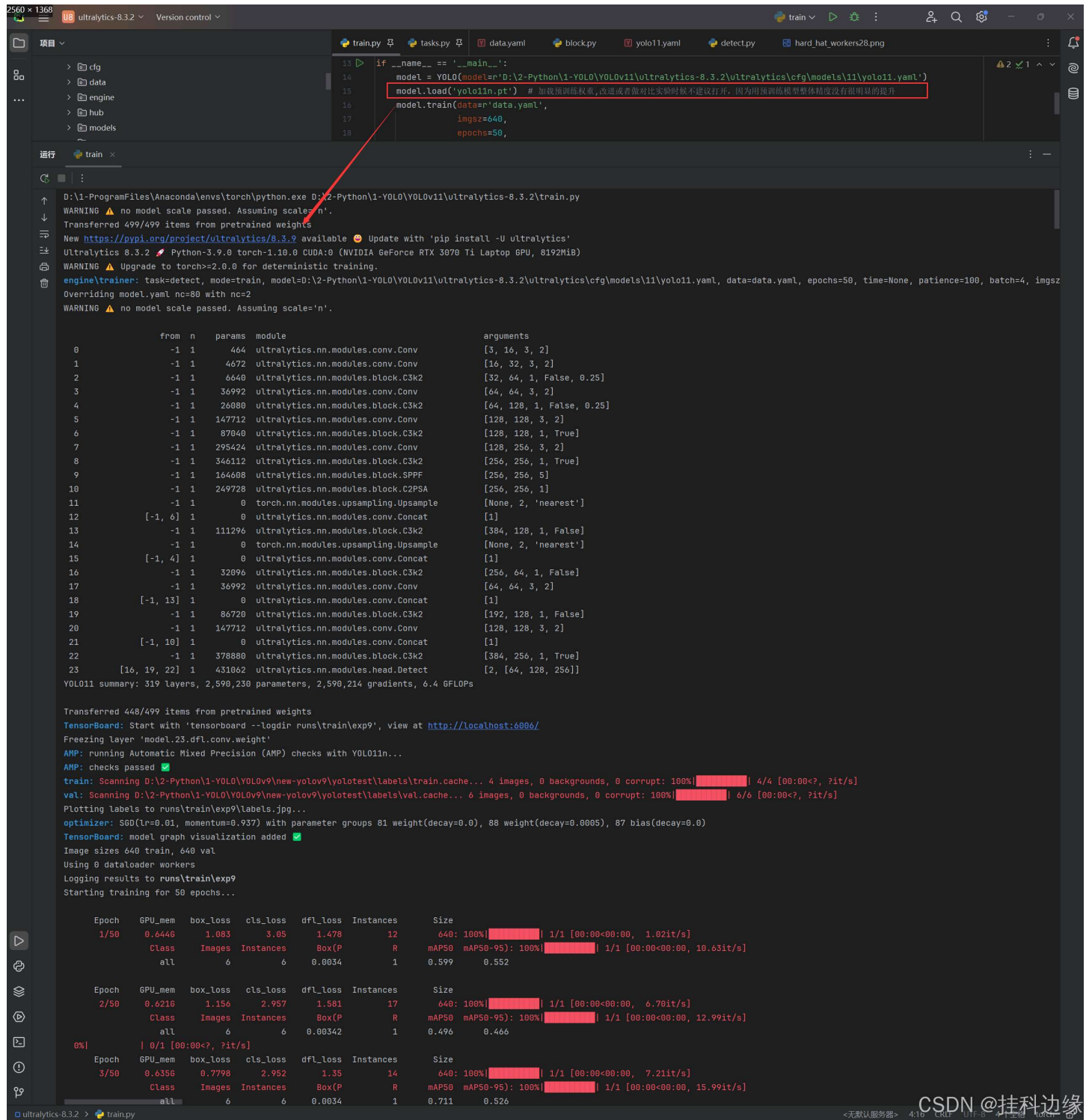
训练代码的参数解释:

- model参数: 该参数填入模型配置文件的路径, 改进的话建议不需要填预训练模型权重
- data参数: 该参数可以填入训练数据集配置文件的路径
- imgs参数: 该参数代表输入图像的尺寸, 指定为 640x640 像素
- epochs参数: 该参数代表训练的轮数
- batch参数: 该参数代表批处理大小, 电脑显存越大, 就设置越大, 根据自己电脑性能设置
- workers参数: 该参数代表数据加载的工作线程数, 出现显存爆了的话可以设置为0, 默认是8
- device参数: 该参数代表用哪个显卡训练, 留空表示自动选择可用的GPU或CPU
- optimizer参数: 该参数代表优化器类型
- close_mosaic参数: 该参数代表在多少个 epoch 后关闭 mosaic 数据增强
- resume参数: 该参数代表是否从上一次中断的训练状态继续训练。设置为False表示从头开始新的训练。如果设置为True, 则会加载上一次训练的模型权重和优化器状态, 继续训练。这在训练被中断或在已有模型的基础上进行进一步训练时非常有用。
- project参数: 该参数代表项目文件夹, 用于保存训练结果

- name参数: 该参数代表命名保存的结果文件夹
- single_cls参数: 该参数代表是否将所有类别视为一个类别, 设置为False表示保留原有类别
- cache参数: 该参数代表是否缓存数据, 设置为False表示不缓存。

注意注意注意: 一般做科研改进工作时候可以不用预训练权重, 因为用预训练模型整体精度很难提高

我这里演示加载预训练权重, 训练输出如下所示:



```
if __name__ == '__main__':
    model = YOLO(model='D:\2-Python\1-YOLO\YOLOv11\ultralytics-8.3.2\ultralytics\cfg\models\11\yolo11.yaml')
    model.load('yolo11n.pt') # 加载预训练权重, 改进或者做对比实验时候不建议打开, 因为用预训练模型整体精度没有明显的提升
    model.train(data='data.yaml',
                imgsiz=640,
                epochs=50,
```

```
Transferred 499/499 items from pretrained weights
New https://pypi.org/project/ultralytics/8.3.9 available Update with 'pip install -U ultralytics'
Ultralytics 8.3.2 Python-3.9.0 torch-1.10.0 CUDA:0 (NVIDIA GeForce RTX 3070 Ti Laptop GPU, 8192MiB)
WARNING Upgrade to torch>=2.0.0 for deterministic training.
engine\trainer: task=detect, mode=train, model=D:\2-Python\1-YOLO\YOLOv11\ultralytics-8.3.2\ultralytics\cfg\models\11\yolo11.yaml, data=data.yaml, epochs=50, time=None, patience=100, batch=4, imgsiz
Overriding model.yaml nc=80 with nc=2
WARNING no model scale passed. Assuming scale='n'.

from n  params  module  arguments
0      -1  1      464    ultralytics.nn.modules.conv.Conv  [3, 16, 3, 2]
1      -1  1      4672   ultralytics.nn.modules.conv.Conv  [16, 32, 3, 2]
2      -1  1      6640   ultralytics.nn.modules.block.C3k2 [32, 64, 1, False, 0.25]
3      -1  1      36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
4      -1  1      26080  ultralytics.nn.modules.block.C3k2 [64, 128, 1, False, 0.25]
5      -1  1      147712 ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
6      -1  1      87040  ultralytics.nn.modules.block.C3k2 [128, 128, 1, True]
7      -1  1      295424 ultralytics.nn.modules.conv.Conv  [128, 256, 3, 2]
8      -1  1      346112 ultralytics.nn.modules.block.C3k2 [256, 256, 1, True]
9      -1  1      164608 ultralytics.nn.modules.block.SPPF [256, 256, 5]
10     -1  1      249728 ultralytics.nn.modules.block.C2PSA [256, 256, 1]
11     -1  1      0      torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
12     [-1, 6] 1      0      ultralytics.nn.modules.conv.Concat [1]
13     -1  1      111296 ultralytics.nn.modules.block.C3k2 [384, 128, 1, False]
14     -1  1      0      torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']
15     [-1, 4] 1      0      ultralytics.nn.modules.conv.Concat [1]
16     -1  1      32096  ultralytics.nn.modules.block.C3k2 [256, 64, 1, False]
17     -1  1      36992  ultralytics.nn.modules.conv.Conv  [64, 64, 3, 2]
18     [-1, 13] 1      0      ultralytics.nn.modules.conv.Concat [1]
19     -1  1      86720  ultralytics.nn.modules.block.C3k2 [192, 128, 1, False]
20     -1  1      147712 ultralytics.nn.modules.conv.Conv  [128, 128, 3, 2]
21     [-1, 10] 1      0      ultralytics.nn.modules.conv.Concat [1]
22     -1  1      378880 ultralytics.nn.modules.block.C3k2 [384, 256, 1, True]
23     [16, 19, 22] 1  431062 ultralytics.nn.modules.head.Detect [2, [64, 128, 256]]

YOLO11 summary: 319 layers, 2,590,230 parameters, 2,590,214 gradients, 6.4 GFLOPs

Transferred 448/499 items from pretrained weights
TensorBoard: Start with 'tensorboard --logdir runs\train\exp9', view at http://localhost:6006/
Freezing layer 'model.23.dfl.conv.weight'
AMP: running Automatic Mixed Precision (AMP) checks with YOLO11n...
AMP: checks passed
train: Scanning D:\2-Python\1-YOLO\YOLOv9\new-yolov9\yolotest\labels\train.cache... 4 images, 0 backgrounds, 0 corrupt: 100%| 4/4 [00:00<, ?it/s]
val: Scanning D:\2-Python\1-YOLO\YOLOv9\new-yolov9\yolotest\labels\val.cache... 6 images, 0 backgrounds, 0 corrupt: 100%| 6/6 [00:00<, ?it/s]
Plotting labels to runs\train\exp9\labels.jpg...
optimizer: SGD(lr=0.01, momentum=0.937) with parameter groups 81 weight(decay=0.0), 88 weight(decay=0.0005), 87 bias(decay=0.0)
TensorBoard: model graph visualization added
Image sizes 640 train, 640 val
Using 0 dataloader workers
Logging results to runs\train\exp9
Starting training for 50 epochs...

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
1/50    0.644G    1.083     3.05     1.478     12         640: 100%| 1/1 [00:00<00:00, 1.02it/s]
Class  Images  Instances  Box(P  R  mAP50  mAP50-95): 100%| 1/1 [00:00<00:00, 10.63it/s]
all      6         6     0.0034  1  0.599  0.552

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
2/50    0.621G    1.156     2.957     1.581     17         640: 100%| 1/1 [00:00<00:00, 6.76it/s]
Class  Images  Instances  Box(P  R  mAP50  mAP50-95): 100%| 1/1 [00:00<00:00, 12.99it/s]
all      6         6     0.00342  1  0.496  0.466

Epoch  GPU_mem  box_loss  cls_loss  dfl_loss  Instances  Size
3/50    0.635G    0.7798     2.952     1.35     14         640: 100%| 1/1 [00:00<00:00, 7.21it/s]
Class  Images  Instances  Box(P  R  mAP50  mAP50-95): 100%| 1/1 [00:00<00:00, 15.99it/s]
all      6         6     0.0034  1  0.711  0.526
```

五、YOLOv11推理

- (1) 官网的预训练模型下载

GitHub - ultralytics/ultralytics: NE x +

https://github.com/ultralytics/ultralytics

README Code of conduct AGPL-3.0 license Security

See YOLO [Python Docs](#) for more examples.

Models

YOLO11 [Detect](#), [Segment](#) and [Pose](#) models pretrained on the [COCO](#) dataset are available here, as well as YOLO11 [Classify](#) models pretrained on the [ImageNet](#) dataset. [Track](#) mode is available for all Detect, Segment and Pose models.

[Classify](#) [Detect](#) [Segment](#) [Track](#) [Pose](#)



All [Models](#) download automatically from the latest Ultralytics [release](#) on first use.

- ▶ Detection (COCO)
- ▶ Segmentation (COCO)
- ▶ Classification (ImageNet)
- ▶ Pose (COCO)
- ▶ OBB (DOTAv1)

Integrations

Our key integrations with leading AI platforms extend the functionality of Ultralytics' offerings, enhancing tasks like dataset labeling, training, visualization, and model management. Discover how Ultralytics, in collaboration with [Roboflow](#), [ClearML](#), [Comet](#), [Neural Magic](#) and [OpenVINO](#), can optimize your AI workflow.

目标检测模型在这里下载

分割模型在这里下载

分类模型这这里下载

关键点检测模型在这里下载

旋转框目标检测模型在这里下载

CSDN @挂科边缘

进入官网的源码下载地址：官网模型下载地址，往下面拉，看到模型位置，YOLOv11 针对不同的场景和应用提供了 YOLOv11n、YOLOv11s 等不同大小的模型，具体看官网提供的，需要下载哪个，鼠标左键单击下载就行。

GitHub - ultralytics/ultralytics: NE x +

https://github.com/ultralytics/ultralytics?tab=readme-ov-file

README Code of conduct AGPL-3.0 license Security

All [Models](#) download automatically from the latest Ultralytics [release](#) on first use.

▼ Detection (COCO)

See [Detection Docs](#) for usage examples with these models trained on [COCO](#), which include 80 pre-trained classes.

Model	size (pixels)	mAP ^{val} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n	640	39.5	56.1 ± 0.8	1.5 ± 0.0	2.6	6.5
YOLO11s	640	47.0	90.0 ± 1.2	2.5 ± 0.0	9.4	21.5
YOLO11m	640	51.5	183.2 ± 2.0	4.7 ± 0.1	20.1	68.0
YOLO11l	640	53.4	238.6 ± 1.4	6.2 ± 0.1	25.3	86.9
YOLO11x	640	54.7	462.8 ± 6.7	11.3 ± 0.2	56.9	194.9

- mAP^{val} values are for single-model single-scale on [COCO val2017](#) dataset. Reproduce by `yolo val detect data=coco.yaml device=0`
- Speed averaged over COCO val images using an [Amazon EC2 P4d](#) instance. Reproduce by `yolo val detect data=coco.yaml batch=1 device=0|cpu`

▼ Segmentation (COCO)

See [Segmentation Docs](#) for usage examples with these models trained on [COCO-Seg](#), which include 80 pre-trained classes.

Model	size (pixels)	mAP ^{box} 50-95	mAP ^{mask} 50-95	Speed CPU ONNX (ms)	Speed T4 TensorRT10 (ms)	params (M)	FLOPs (B)
YOLO11n-seg	640	38.9	32.0	65.9 ± 1.1	1.8 ± 0.0	2.9	10.4
YOLO11s-seg	640	46.6	37.8	117.6 ± 4.9	2.9 ± 0.0	10.1	35.5

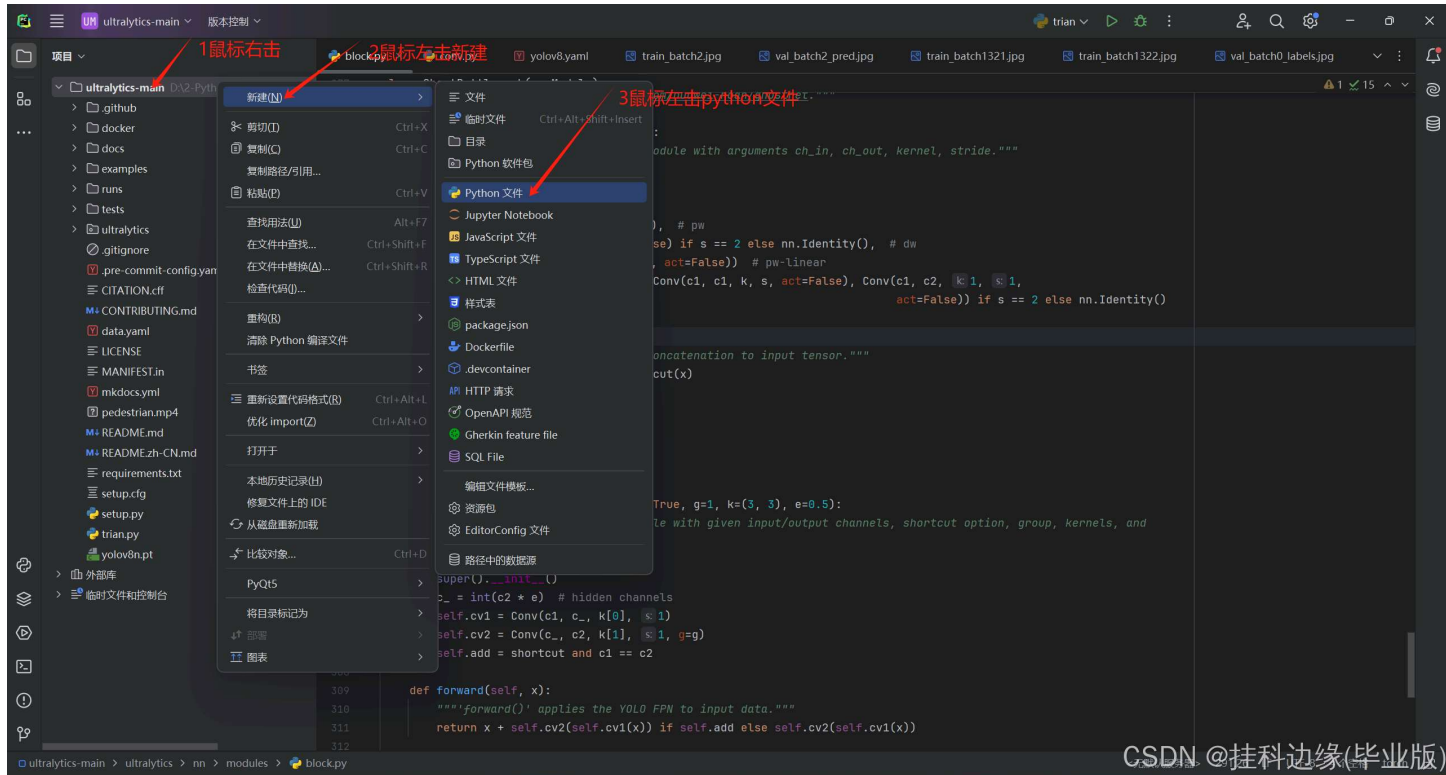
我下载目标检测

我也下载一个分割模型

CSDN @挂科边缘

我的源码包已经下载好了模型了，如果需要其他权重自行下载就行

(2) 在根目录新建一个python文件，取名为：detect.py



(3) 把推理代码复制到detect.py文件

注意注意注意：模型路径改成你自己的路径，还有预测图像也改成你自己的路径

推理的代码如下：

```
1 # -*- coding: utf-8 -*-
2 """
3 @Auth : 挂科边缘
4 @File : detect.py
5 @IDE : PyCharm
6 @Motto:学习新思想，争做新青年
7 @Email : 179958974@qq.com
8 """
9
10 from ultralytics import YOLO
11
12 if __name__ == '__main__':
13
14     # Load a model
15     model = YOLO(model=r'D:\2-Python\1-YOLO\YOLOv11\ultralitics-8.3.2\yolo11n-seg.pt')
16     model.predict(source=r'D:\2-Python\1-YOLO\YOLOv11\ultralitics-8.3.2\ultralitics\assets\bus.jpg',
17                   save=True,
18                   show=True,
19                   )
```

推理代码的参数解释

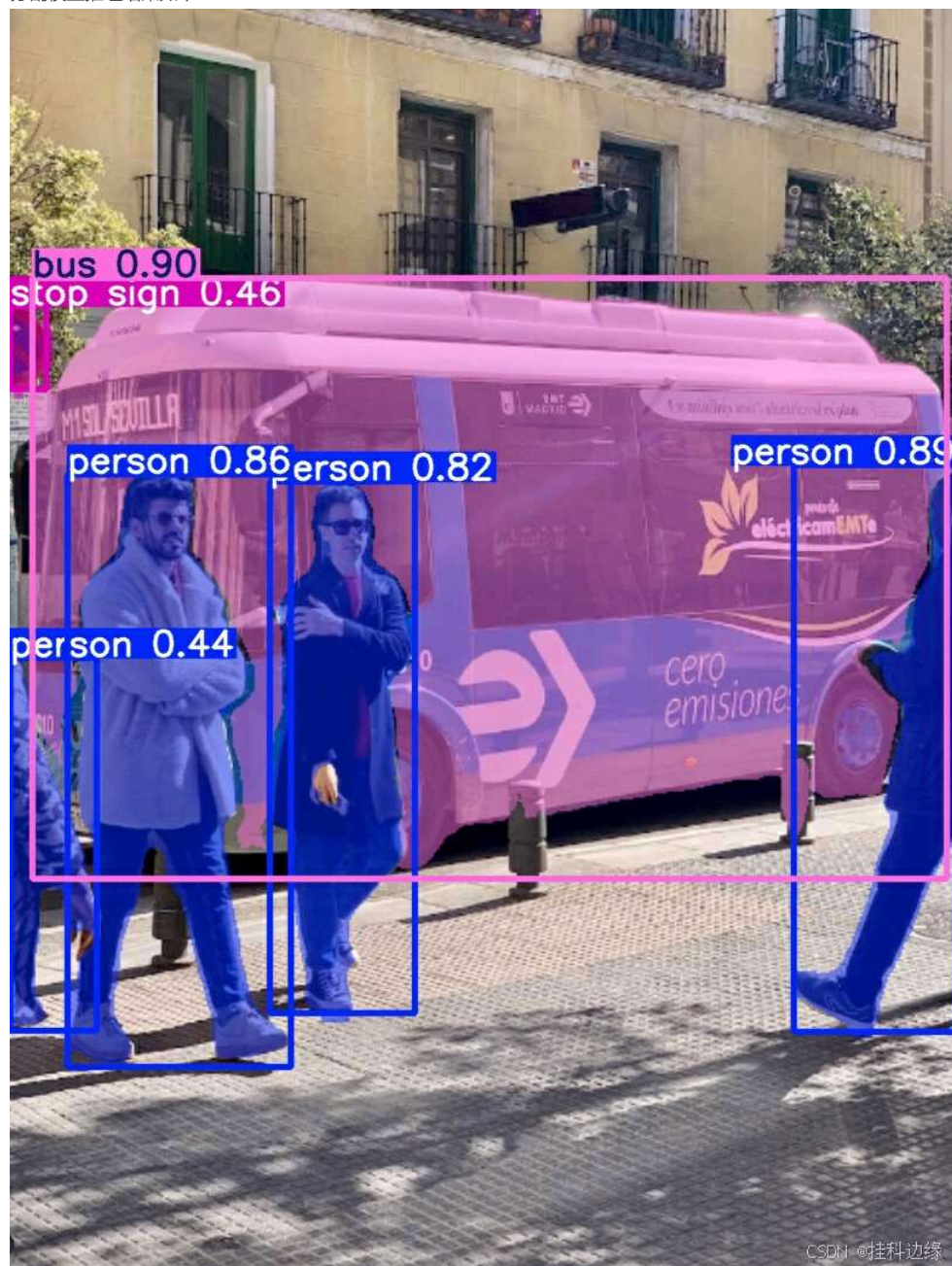
1.model参数：该参数可以填入模型文件路径

2.source参数：该参数可以填入需要推理的图片或者视频路径，如果打开摄像头推理则填入0就行

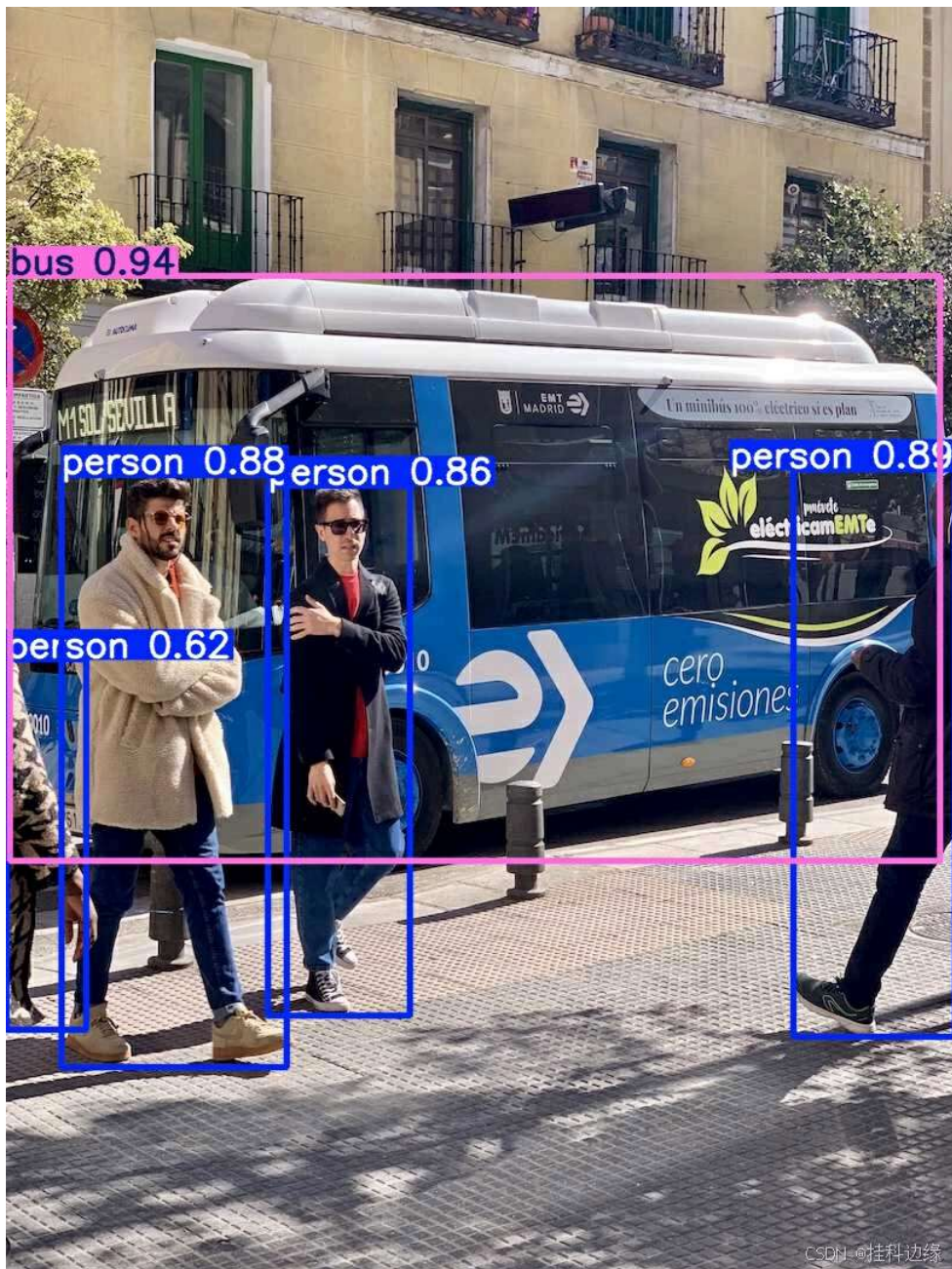
3.save参数：该参数填入True，代表把推理结果保存下来，默认是不保存的，所以一般都填入True

4.show参数：该参数填入True，代表把推理结果以窗口形式显示出来，默认是显示的，这个参数根据自己需求打开就行，不显示你就填False就行

分割模型推理结果如下:

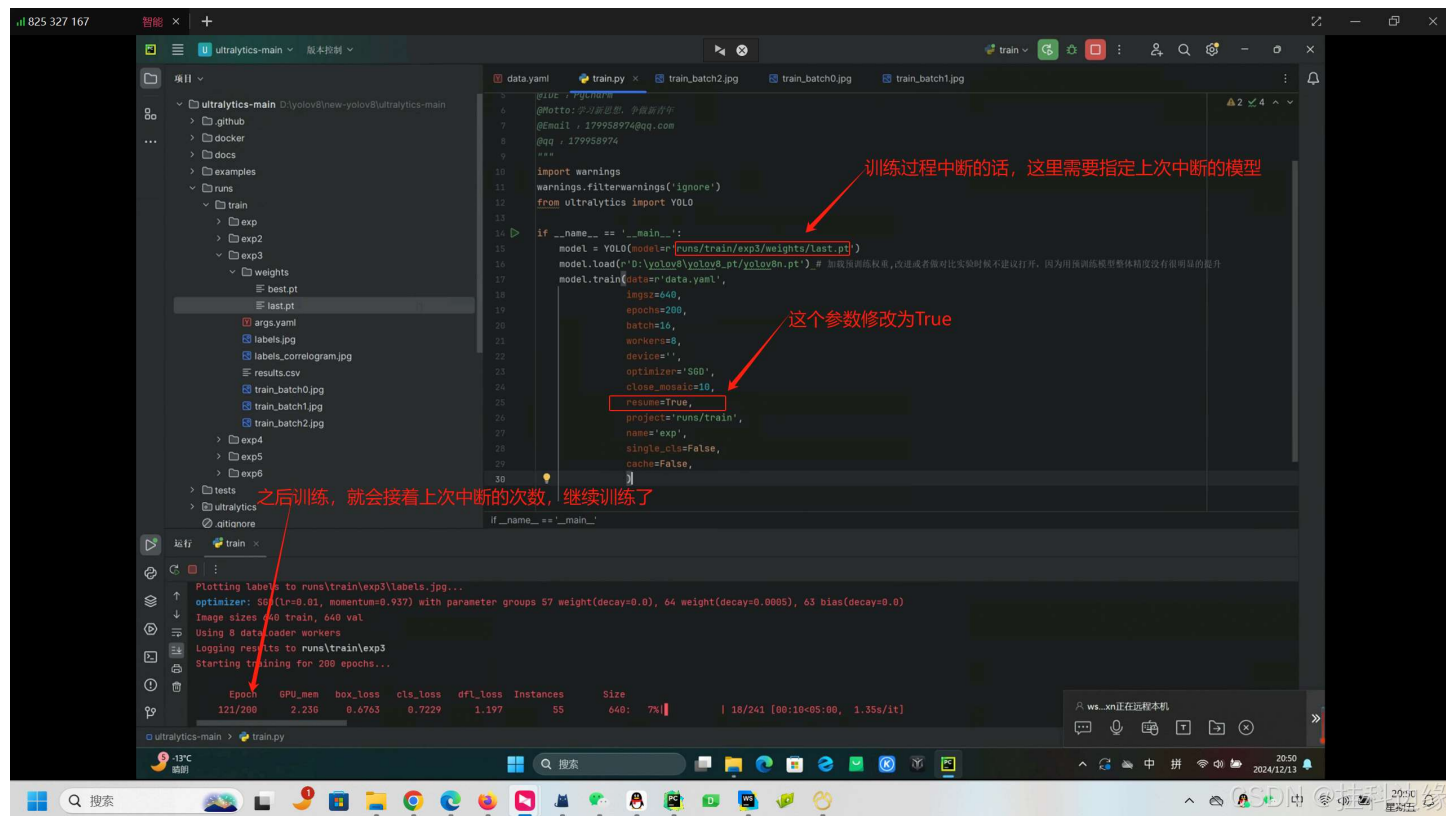


目标检测模型 推理结果如下：



六、解决训练过程中断怎么继续上次训练

在训练过程不小心中断了，那怎么继续上次的训练了，这里先不慌，官网也的代码写得非常好，它有这个断点训练功能，那么 YOLOv8 v10 v11 处理的方法都是一模一样，接下来直接看图操作就行：



- model参数: 该参数填入上次中断的模型, 为 last.pt
- resume参数: 该参数设置为True, 则会加载上一次训练的模型权重和优化器状态, 继续训练。

总结

YOLOv11训练自己数据集和推理到此结束, 有问题可以留言, 创作不易, 请帮忙点个赞, 谢谢

